MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Daniel Moisei FAF-241

Report

Laboratory Work No. 5

Operating Systems

Topic: The File System with Your Own Hands

Checked by:
Reițman Patricia, *university assistant*

Chișinău - 2026

CONTENTS

Introduction **3**

1 The Tree and the File Types **4**

1.1 Root hierarchy and path types 4

1.2 Observations 5

2 Inodes and Links **6**

2.1 Hard and symbolic links 6

2.2 Directory links and sparse allocation 7

3 Permissions **9**

3.1 File modes and umask 9

3.2 Directory permissions and special bits 9

4 Disks and the Kernel as Files **11**

4.1 Storage layout and root utilization 11

4.2 CPU, memory, process identity, and descriptors 11

5 Optional Experiment: Magic Numbers **13**

Conclusion **14**

References **15**

INTRODUCTION

The purpose of this laboratory work was to examine the Linux file system through direct experiments with the directory tree, file types, inodes, links, permissions, disks, and the kernel interfaces exposed through `/proc`. All commands were executed as the ordinary user `moiseid`; the experiment therefore exercised real permission checks rather than the exemptions available to `root`.

The tests were run on CachyOS Linux (Arch-based) with a Btrfs root file system. They showed that file names are directory entries referring to inodes, hard links share an inode, symbolic links store a path, permissions are evaluated independently for files and directories, and sparse logical size is different from allocated disk space. System inspection identified a 953.9 GB NVMe disk, a 457.4 GB Btrfs root partition, 16 logical processors, and approximately 14.8 GiB of RAM. The report includes screenshots of the actual terminal sessions.

1 THE TREE AND THE FILE TYPES

1.1 Root hierarchy and path types

```
ls /
ls -ld /etc /etc/passwd /bin /dev/null /tmp
ls -l /dev | head
file /usr/bin/ls /etc/passwd /bin
du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
```

The first character printed by `ls -l` identifies the object type. In the observed output, `/etc` and `/tmp` begin with `d`, so they are directories; `/etc/passwd` begins with `-`, so it is a regular file; `/bin` begins with `l`, so it is a symbolic link; and `/dev/null` begins with `c`, so it is a character device. The `file` utility independently classified `/usr/bin/ls` as a 64-bit ELF executable, `/etc/passwd` as ASCII text, and `/bin` as a symbolic link.

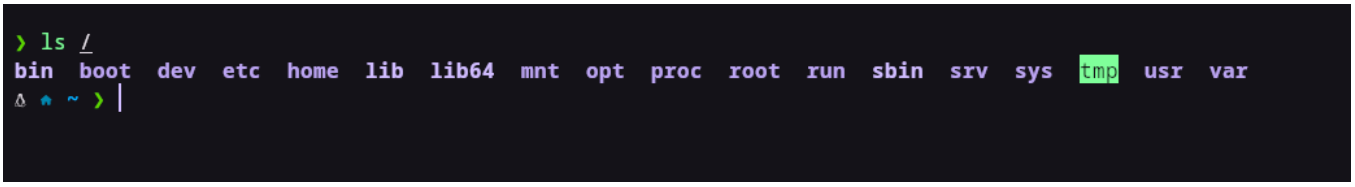


Figure 1.1: The top-level Linux directory tree

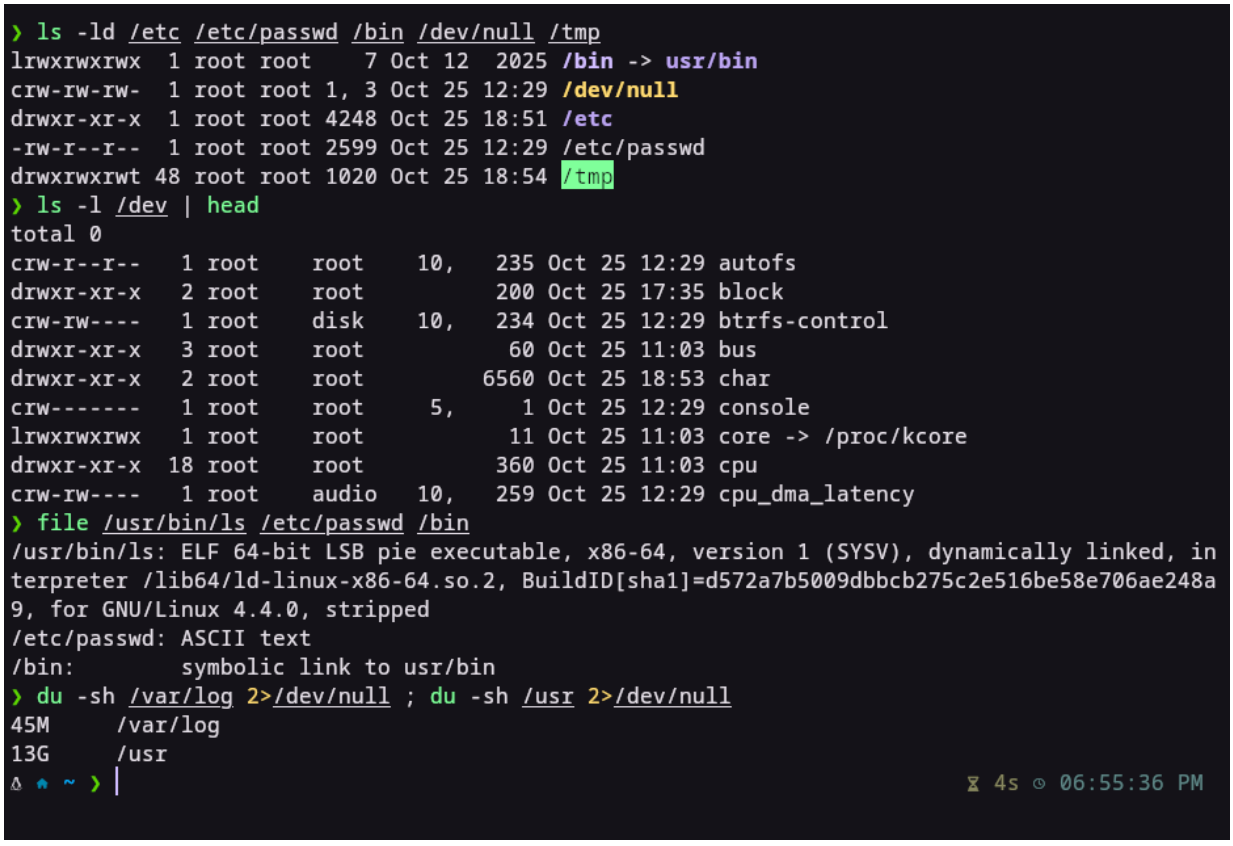


Figure 1.2: File types, device entries, and directory disk usage

1.2 Observations

On this system, `/bin` is not a separate directory: it is a relative symbolic link to `usr/bin`, which resolves to `/usr/bin`. A character-device example is `/dev/null`, with major and minor numbers `1,3`. A block-device example is `/dev/nvme0n1`, whose listing begins with `b` (`brw-rw----`, major and minor numbers `259,0`, group `disk`); its partition `/dev/nvme0n1p6` is also a block device (`259,6`). The commands `lsblk`, `df`, and `mount` consistently identify `nvme0n1` as the physical block device and `nvme0n1p6` as the root partition.

The measured size of `/var/log` was 45 MB, while `/usr` occupied 13 GB. Therefore `/usr` was larger by about 13 GB, or roughly three hundred times. These values describe the state of this machine at the time of execution and may change as packages and logs change.

2 INODES AND LINKS

2.1 Hard and symbolic links

```
echo hello > notes.txt
stat notes.txt
ls -li notes.txt
ln notes.txt hard.txt
ls -li notes.txt hard.txt
rm notes.txt
cat hard.txt ; ls -li hard.txt
ln -s hard.txt soft.txt
ls -li hard.txt soft.txt
cat soft.txt
rm hard.txt
cat soft.txt ; ls -l soft.txt
```

The `stat` result reports the inode in the `Inode` field and the number of directory names in the `Links` field. Initially, `notes.txt` used inode `601972` and had one link. After `ln notes.txt hard.txt`, both names displayed the same inode and the link count became two.

Removing `notes.txt` only removed that directory entry and reduced the inode's link count. The name `hard.txt` still referred to the same inode, so its data remained readable. In general, `rm` unlinks a name; the inode and its data can be reclaimed only after the link count reaches zero and no process still has the file open.

The symbolic link used a separate inode, `602007`, and stored the target text `hard.txt`. After that target was removed, `soft.txt` still existed as an eight-byte symbolic link, but path resolution ended at a missing name and `cat` failed. Its size was exactly eight bytes because the stored path string `hard.txt` contains eight characters.

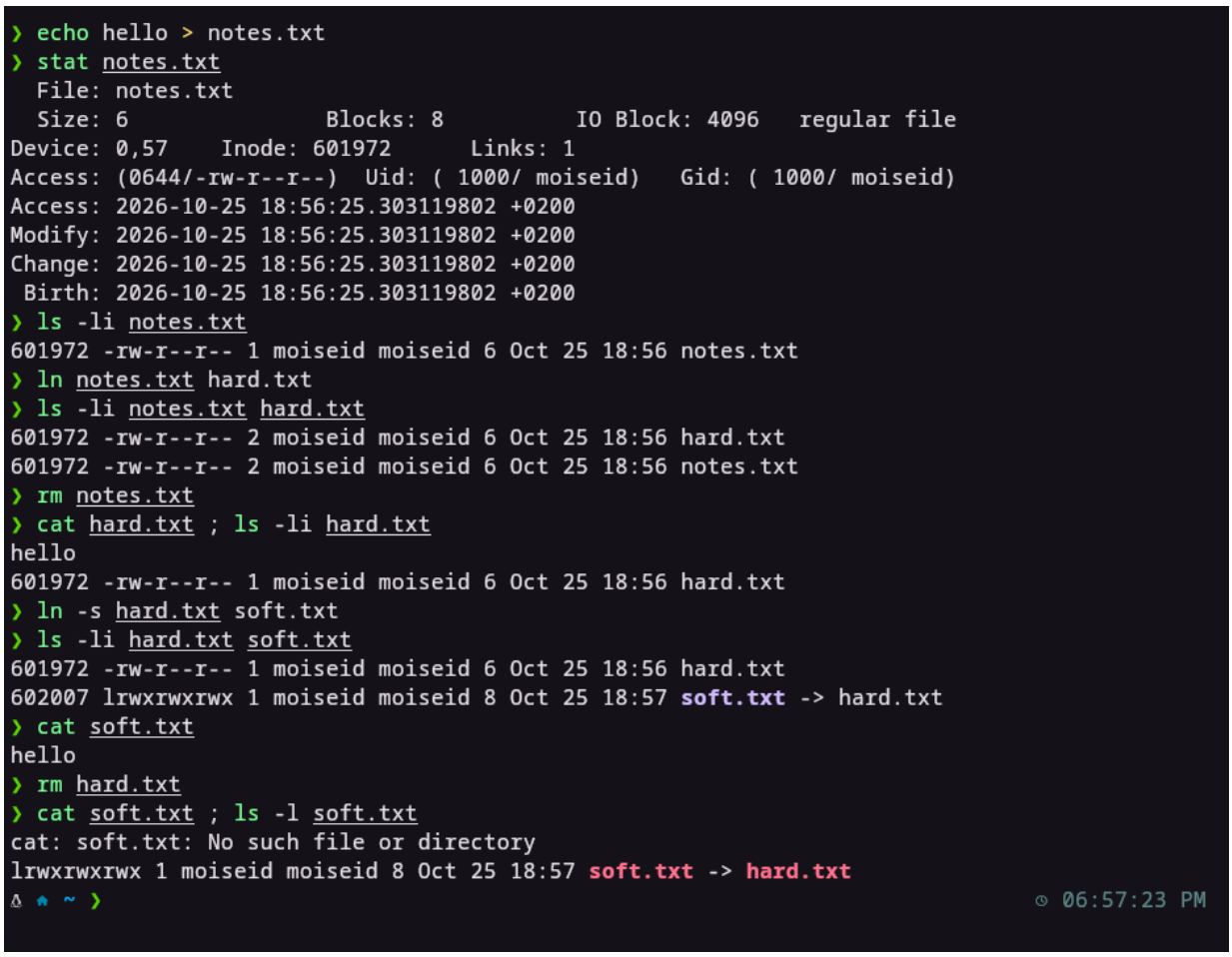


Figure 2.1: Inode numbers, link counts, and removal behavior

2.2 Directory links and sparse allocation

On a traditional Unix file system, a new directory has link count two: one name in its parent and its own `.` entry. Each immediate subdirectory contributes one additional reference through its `..` entry, so creating `d/x` and `d/y` conceptually raises the count from two to four. The captured Btrfs file system reports a fixed directory link count of one both before and after the subdirectories are created; this is a Btrfs reporting behavior, not a change to the meaning of `.` and `..` during path traversal.

The command `echo x > one.txt` creates two logical bytes: the letter `x` and a newline. Nevertheless, `du` reported 4.0 KB because the file occupies a file-system allocation unit. Conversely, `truncate -s 1G big.img` established a logical length of 1.0 GB without writing data blocks. It is a sparse file, so `ls` reported 1.0 GB while `du` reported zero allocated space.

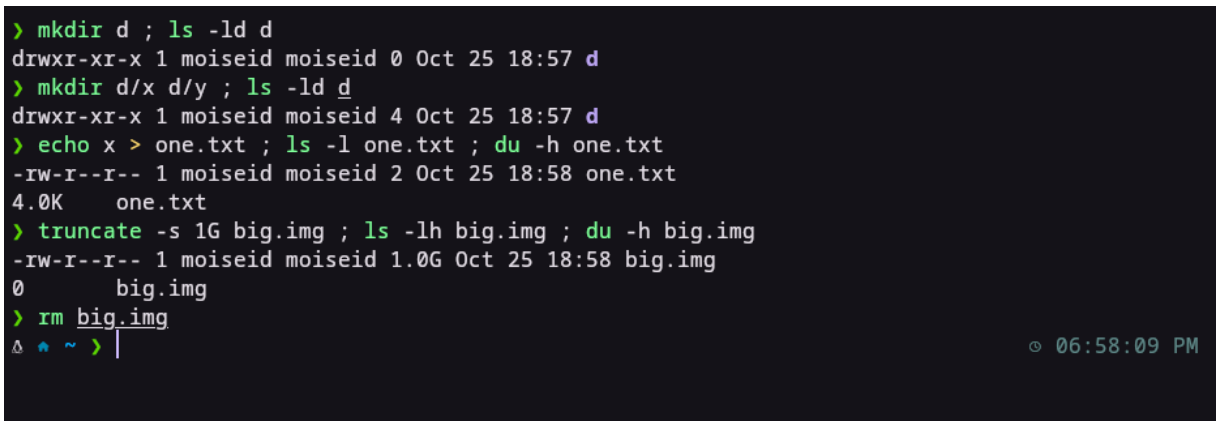


Figure 2.2: Directory link counts and the difference between size and allocated space

3 PERMISSIONS

3.1 File modes and umask

The requested numeric modes correspond to the following permission strings and typical uses:

Mode	Symbolic form	Example use
600	`rw-------`	A private secret or SSH private key readable and writable only by its owner.
644	`rw-r--r--`	A normal document that its owner edits and other users may read.
755	`rwxr-xr-x`	A public executable or searchable directory that only its owner may modify.
700	`rwx------`	A private directory accessible only to its owner.

Ownership selects which permission triplet is checked; it does not bypass the bits in that triplet. Therefore, after `chmod 000 s.txt`, the owner was still `moiseid` but the owner's read bit was clear and `cat` failed. Restoring `u+r` made the content readable again.

The observed umask was `0022`. A newly created regular file starts from mode 666 and loses the masked write bits, producing 644 (`rw-r--r--`). A new directory starts from mode 777 and loses the same bits, producing 755 (`rwxr-xr-x`). The screenshot confirms both results.



Figure 3.1: File permissions, ownership checks, and umask results

3.2 Directory permissions and special bits

Read permission on a directory allows its entry names to be listed. Search or execute permission permits traversal to a known entry and access to its metadata or contents, subject to that entry's own mode. Consequently, with read-only permission on `box`, `ls box` printed `f` (together with an error, because the colour-enabled `ls` also tried to read the metadata of `box/f`) but `cat box/f` failed because path traversal was forbidden. With execute-only permission, listing failed but `cat box/f` succeeded because the known name could be traversed and the file itself was readable.

The `s` in `-rwsr-xr-x` is the set-user-ID bit combined with owner execute permission. Here `passwd` is owned by `root`; the bit lets it run with the owner's effective identity so it can perform the narrowly controlled update to protected password data. The denial from `cat /etc/shadow` confirms that an ordinary process cannot read the protected file directly.

The `t` in `drwxrwxrwt` is the sticky bit. Although all users may create files in `/tmp`, it prevents one ordinary user from deleting or renaming an entry owned by another user. This allows a shared writable directory without permitting arbitrary removal of other users' temporary files.

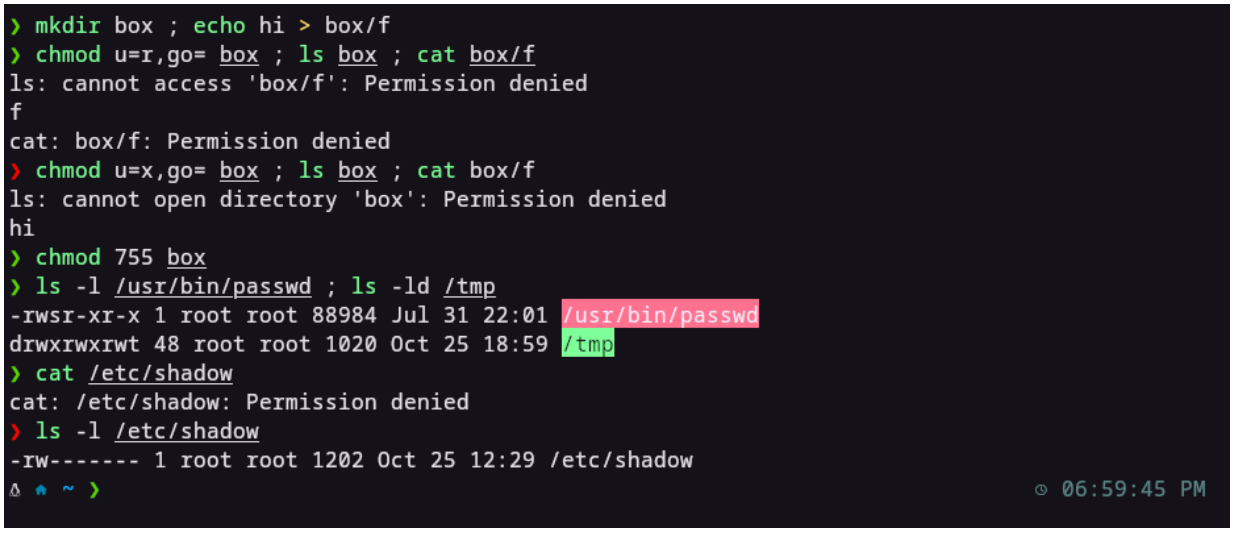


Figure 3.2: Directory read and search permissions, setuid, sticky bit, and shadow protection

4 DISKS AND THE KERNEL AS FILES

4.1 Storage layout and root utilization

The physical storage tree observed with `lsblk` was:

```
nvme0n1                  953.9G disk
|-nvme0n1p1                200M vfat
|-nvme0n1p2                 16M
|-nvme0n1p3              491.3G ntfs
|-nvme0n1p4                889M ntfs
|-nvme0n1p5                  4G vfat   /boot
`-nvme0n1p6              457.4G btrfs  /
```

The root file system is Btrfs on `/dev/nvme0n1p6`. At measurement time, `df -hT /` reported 458 GB total, 63 GB used, 395 GB available, and 14% utilization. The `mount` output confirmed Btrfs and showed the active compression, SSD, discard, and subvolume options.

```
❯ lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
NAME          SIZE TYPE FSTYPE MOUNTPOINTS
zram0        14.8G disk swap   [SWAP]
nvme0n1     953.9G disk
├─nvme0n1p1   200M part vfat
├─nvme0n1p2    16M part
├─nvme0n1p3 491.3G part ntfs
├─nvme0n1p4   889M part ntfs
├─nvme0n1p5     4G part vfat   /boot
└─nvme0n1p6 457.4G part btrfs  /var/tmp
                               /var/cache
                               /var/log
                               /srv
                               /home
                               /root
                               /
❯ df -hT /
Filesystem     Type   Size  Used Avail Use% Mounted on
/dev/nvme0n1p6 btrfs  458G   63G  395G  14% /
❯ mount | grep " / "
/dev/nvme0n1p6 on / type btrfs (rw,noatime,compress=zstd:1,ssd,discard=async,space_cache=v2,
subvolid=256,subvol=/@)
 ~ ❯                                                                    07:00:24 PM
```

Figure 4.1: NVMe partitions, the Btrfs root file system, and its utilization

4.2 CPU, memory, process identity, and descriptors

The count from `/proc/cpuinfo` was 16 logical processors. The total-memory line in `/proc/meminfo` reported 15,566,504 kB, which is about 15.6 GB in decimal units or 14.85 GiB. Free and available memory are dynamic and therefore describe only the instant at which the command ran.

The first line of `/proc/self/status` was `Name: head`. The path `/proc/self` is resolved separately for the process that opens it. Although the shell launched the command, the `head` process opened and read the file, so `self` referred to `head`. Similarly, `ls -l /proc/self/fd` displayed the descriptors belonging to the `ls` process: standard input, output, error, and the descriptor used while reading its own file-descriptor directory.

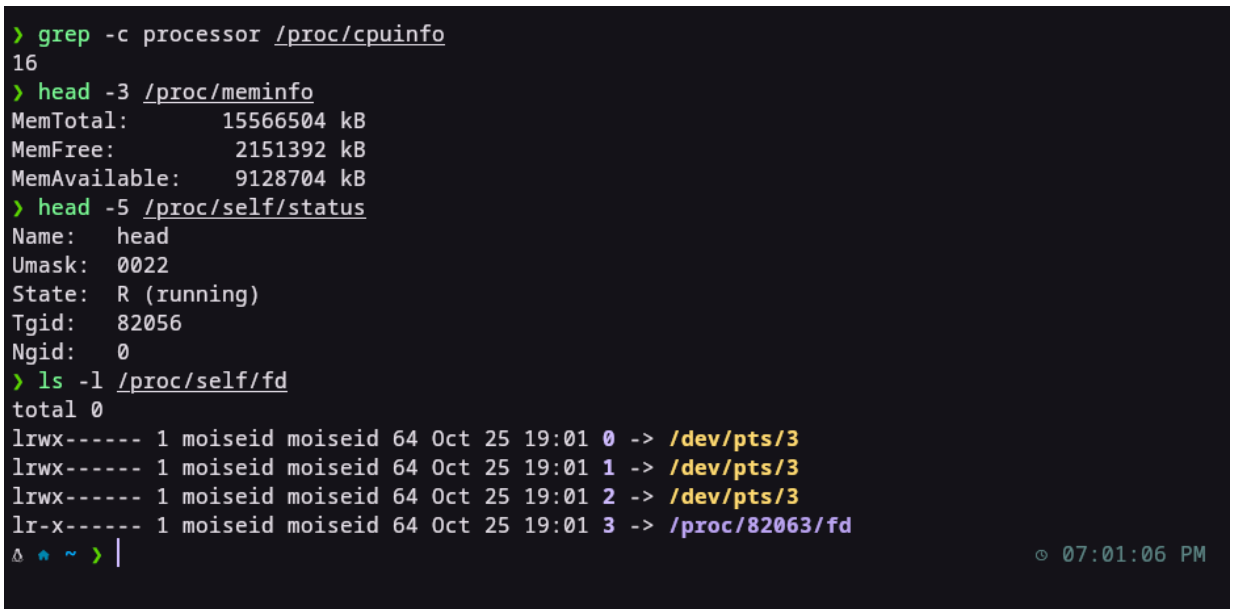


Figure 4.2: Processor count, memory information, process status, and open descriptors

5 OPTIONAL EXPERIMENT: MAGIC NUMBERS

```
gzip -c /etc/passwd > compressed-notes.txt
file compressed-notes.txt
od -An -tx1 -N16 compressed-notes.txt
```

The output file deliberately retained the misleading `.txt` suffix, yet `file` identified gzip-compressed data. Its first bytes were `1f 8b 08`: `1f 8b` is the gzip magic number and `08` selects the DEFLATE compression method. The classifier therefore relied on the file header rather than its name or extension. The available `od` utility was used as the byte-display equivalent of `xxd`.

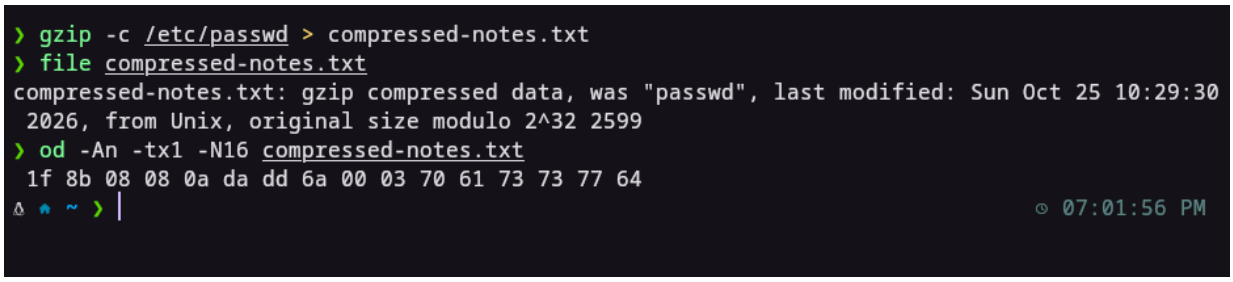


Figure 5.1: A gzip signature detected despite a text-file extension

CONCLUSION

This laboratory connected visible path names to the lower-level file-system objects that Linux manages. The hard-link experiment proved that multiple names can refer to one inode and that removing one name does not necessarily remove the data. The symbolic-link experiment showed the opposite structure: the link has its own inode and stores a path that may become invalid. Directory link-count behavior also exposed a useful implementation detail: the traditional 2-plus-subdirectories model explains Unix semantics even though Btrfs reports a fixed count of one.

The permission tests separated ownership from authorization and separated directory listing from directory traversal. They also demonstrated how umask derives default file and directory modes, why setuid is needed for controlled privileged operations such as password changes, and why the sticky bit is essential in shared directories. Finally, `lsblk`, `df`, `mount`, and `/proc` connected the file-oriented interface to real storage, processors, memory, processes, and descriptors. The sparse-file and magic-number experiments reinforced that logical size and filename extensions are only metadata views; actual allocation and content signatures must be inspected directly.

REFERENCES

[1] Operating Systems Laboratory Work No. 5. *The File System with Your Own Hands*. FCIM / FAF / UTM, 2026-2027.

[2] Linux manual pages for `stat(1)`, `ln(1)`, `chmod(1)`, `umask(2)`, `lsblk(8)`, `df(1)`, and `proc(5)`, as installed on the test system.